

Character-Based Sequence-to-Sequence Modelling for Ukrainian Grammatical Error Correction

Viktoriia Tsosenko

Heinrich Heine University Düsseldorf
Viktoriia.Tsosenko@hhu.de

Abstract

Grammatical error correction (GEC) has been studied successfully for English but remains a challenging task for morphologically complex languages such as Ukrainian. In this work, we present a character-level sequence-to-sequence model based on a bidirectional LSTM encoder and an LSTM decoder for Ukrainian GEC. We train and evaluate our model on the manually annotated UA-GEC dataset, consisting of 33,735 sentences and covering grammatical, spelling, punctuation, and fluency errors. We then use a fine-tuned mBART50 baseline and compare it against our model.

1 Introduction

Grammatical Error Correction (GEC) in Natural Language Processing (NLP) performs automatic error detection and correction in written texts. The main error types are (Bryant et al., 2017):

- (a) orthographic (spelling and punctuation),
- (b) morphological (word ending or forms),
- (c) syntactic (word order or missing words),
- (d) lexical errors (wrong context or phrasing).

Interest in Ukrainian NLP has grown significantly in recent years. Despite this, Ukrainian remains a low-resource language due to the lack of annotated corpora. In this work we use the first and currently the only dataset for Ukrainian, UA-GEC. Because of its morphological complexity (e.g. deep system of cases, verbal aspects), Ukrainian GEC is a challenging task for token-based models. Unlike English, Ukrainian is an inflectional language. The most common errors in Ukrainian happen on the morphological level (wrong word endings and agreement) and lexical level (calques). That is why we use a character-based approach. We also compare it against a fine-tuned mBART50 baseline

(Tang et al., 2021), since this model achieved the best results in the UA-GEC shared task (Syvokon and Romanyshyn, 2023a).

To summarize, our contributions are as follows:

- Train a character-based encoder-decoder model on UA-GEC and evaluate it with the F0.5 metric.
- Compare our model against the fine-tuned mBART50.
- Analyze performance based on error types and discuss the pros and cons of a character-based approach for a fusional language.

2 Dataset

Ukrainian belongs to low-resource languages and UA-GEC is the first professionally annotated dataset for Ukrainian GEC. It consists of written texts, both from native and non-native speakers, and annotated by two different annotators. The annotation has the following format: {error=>edit::error_type=Tag}, where *error* is the original text, *edit* is the correction, and *Tag* denotes the error category (Syvokon et al., 2023).

Errors in UA-GEC are divided into four types, namely grammar, fluency, spelling and punctuation. There are two corpus versions in m2 format, namely *gec only* and *gec + fluency*. In this paper, we concentrate on the latter one. *gec + fluency* is divided into train and test sets, with 31,038 and 2,697 sentences, respectively (Syvokon et al., 2023).

The m2 format is standard in GEC and its structure is shown in Table 1. Line S contains a sentence and A-lines are annotations stating error location, its type and the corresponding correction, made by two different annotators.

To use the sentences in training, they must be parsed first. UA-GEC repo provides already parsed sentences as source and target text files, so no

Line	Content
S	She go to the store every day .
A	1 2 G/Number goes REQUIRED -NONE- 0
A	1 2 G/Number goes REQUIRED -NONE- 1

Table 1: An example of a m2-formatted sentence.

additional parsing is required (Syvokon and Romanynshyn, 2023a).

3 Models

3.1 Character-Level Encoder-Decoder

As the model’s name suggests, it is a character-level model consisting of two parts, namely the Encoder and Decoder.

3.1.1 Character-Level Vocabulary

A character-level model is the right approach for detecting errors in fusional languages like Ukrainian, since it allows the model to recognize grammatical function of inflections. A word-level model is unsuitable for this task because it treats different inflected forms of the same word (e.g. different cases or moods) as completely different tokens, resulting in a vocabulary of thousands of words.

To build the vocabulary, we used the train dataset of both source and target sentences. As a result, we received 322 distinct characters, of which only 70 are Cyrillic. Although the Ukrainian alphabet consists of 33 letters, the model treats their upper and lower case forms as separate symbols (66 in total), plus there are four Russian-only letters found in the data (Ѡ, ѡ, Ѣ, Ѥ). The remaining characters that explain such a high count are: 61 Latin letters, 16 digits (including superscript) and 175 other symbols such as emojis, punctuation and other signs.

Additionally, there are four special symbols we added at the beginning of the vocabulary so that the model can indicate padding (<PAD>), start of the sentence (<SOS>), end of the sentence(<EOS>), and any unknown symbols (<UNK>). The (<UNK>) symbol is reserved for any characters unseen during training but present in future validation and inference steps. Thus the final vocabulary for Ukrainian GEC contains 326 symbols in total.

The model cannot work with those symbols directly, it requires a numerical input. Thus, every symbol receives its own numerical ID (see Table 2). Each ID is then mapped to a learned embedding vector required by the encoder and decoder.

Token	Index
<PAD>	0
<SOS>	1
<EOS>	2
<UNK>	3
‘a’	4
‘б’	5
‘B’	6
⋮	⋮

Table 2: Token-to-index mapping. Indices 0–3 are reserved for special tokens; character indices begin at 4.

3.1.2 Encoder

The encoder reads erroneous Ukrainian sentences, one character at a time and converts each character into a vector. Each character ID first attaches to a learned embedding vector through an embedding layer. Next, the embeddings are processed by a *single-layer bidirectional LSTM*, meaning it reads each sentence in both directions, left-to-right and right-to-left. This lets the model understand each character better, based not only on the context before the character, but also on what follows. For Ukrainian it plays an important role, since agreement and case endings often rely on the words that come after. There are two outputs that the encoder produces: a reading of every character (for attention mechanism, not used in this paper), and a context vector.

Forward and backward readings form two hidden states that are concatenated into one. Similarly within the cell states, their directions get concatenated and become one cell state. After concatenation the size of both states needs to be reduced to the decoder’s expected size. By applying two separate *linear projections*, followed by a *tanh activation*, we receive the decoder’s starting point.

3.1.3 Decoder

The decoder’s task is to write correct sentences. When it receives the encoder’s summary, it starts predicting correct letters, one character at a time, left-to-right. By reading the special tokens <SOS> and <EOS> the decoder knows when the sentence begins and ends.

During training the decoder makes predictions, but they are never fed back into the model. Instead, *teacher forcing* is used, meaning the model always receives the correct previous characters. This enables a more faster training process.

The inference time is where the decoder uses its own predictions and actually demonstrates what it has learned. After training, the best model is saved and loaded again to correct new sentences at inference time.

3.2 Fine-tuned mBART50

transformer based encoder-decoder pretrained on 50 languages
token-level approach

4 Training

Hyperparameters, hardware etc.

5 Results

Table of comparison

References

- Maksym Bondarenko, Artem Yushko, Andrii Shportko, and Andrii Fedorych. 2023. [Comparative study of models trained on synthetic data for Ukrainian grammatical error correction](#). In *Proceedings of the Second Ukrainian Natural Language Processing Workshop (UNLP)*, pages 103–113, Dubrovnik, Croatia. Association for Computational Linguistics.
- Christopher Bryant, Mariano Felice, and Ted Briscoe. 2017. [Automatic annotation and evaluation of error types for grammatical error correction](#). In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 793–805, Vancouver, Canada. Association for Computational Linguistics.
- Carl James. 1998. *Errors in Language Learning and Use: Exploring Error Analysis*, 1 edition. Routledge, London.
- Satoru Katsumata and Mamoru Komachi. 2020. [Stronger baselines for grammatical error correction using a pretrained encoder-decoder model](#). In *Proceedings of the 1st Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics and the 10th International Joint Conference on Natural Language Processing*, pages 827–832, Suzhou, China. Association for Computational Linguistics.
- Lukasz Kulowski. 2019. LSTM encoder-decoder time series forecasting. https://github.com/lkulowski/LSTM_encoder_decoder. Accessed: 2026-05-08.
- PyTorch Discuss Community. 2018. [How to use collate_fn](#). Accessed: 2026-05-16.
- Oleksiy Syvokon, Olena Nahorna, Pavlo Kuchmiichuk, and Nastasiia Osidach. 2023. [UA-GEC: Grammatical error correction and fluency corpus for the Ukrainian language](#). In *Proceedings of the Second Ukrainian Natural Language Processing Workshop (UNLP)*, pages 96–102, Dubrovnik, Croatia. Association for Computational Linguistics.
- Oleksiy Syvokon and Mariana Romanyshyn. 2023a. UA-GEC: Grammatical Error Correction Corpus for Ukrainian. <https://github.com/grammarly/ua-gec>. Accessed: 2026-04-27.
- Oleksiy Syvokon and Mariana Romanyshyn. 2023b. [The UNLP 2023 shared task on grammatical error correction for Ukrainian](#). In *Proceedings of the Second Ukrainian Natural Language Processing Workshop (UNLP)*, pages 132–137, Dubrovnik, Croatia. Association for Computational Linguistics.
- Yuqing Tang, Chau Tran, Xian Li, Peng-Jen Chen, Naman Goyal, Vishrav Chaudhary, Jiatao Gu, and Angela Fan. 2021. [Multilingual translation from denoising pre-training](#). In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pages 3450–3466, Online. Association for Computational Linguistics.